

Quale delle seguenti affermazioni riguardo all'uso di subquery scalari in MySQL è falsa?

- ☐ a. Una subquery scalare restituisce sempre un unico valore numerico
- ☐ b. Se una subquery scalare restituisce più di un record, MySQL genera un errore
- ☐ c. Nessuna delle altre alternative è falsa
- ☐ d. Una subquery scalare può essere usata in una clausola WHERE, purché restituisca al più un record

Dato lo schema di database usato a lezione:

PAZIENTE(CodFiscale, Nome, Cognome, Sesso, DataNascita, Citta, Reddito)

MEDICO(Matricola, Nome, Cognome, Specializzazione, Citta, Parcella)

VISITA(Paziente, Medico, Data, Mutuata)

Considerata la query q a sinistra, quale delle query a destra è una versione join-equivalente?

1)

```
SELECT DISTINCT P.Nome, P.Cognome
FROM Paziente P
      INNER JOIN
      Visita V ON P.CodFiscale = V.Paziente
WHERE V.Mutuata = 0;
```

Query q :

```
SELECT P.Nome, P.Cognome
FROM Paziente P
WHERE P.CodFiscale NOT IN (
  SELECT Paziente
  FROM Visita
  WHERE Mutuata = 1);
```

2)

```
SELECT DISTINCT P.Nome, P.Cognome
FROM Paziente P
      LEFT OUTER JOIN
      Visita V ON P.CodFiscale = V.Paziente
WHERE V.Mutuata = 0;
```

3)

```
SELECT DISTINCT P.Nome, P.Cognome
FROM Paziente P
      LEFT OUTER JOIN
      Visita V ON P.CodFiscale = V.Paziente
WHERE V.Mutuata = 0
      OR V.Paziente IS NULL;
```

- ☐ a. Nessuna delle altre alternative è corretta
- ☐ b. La query 1) è join-equivalente alla query q
- ☐ c. La query 2) è join-equivalente alla query q
- ☐ d. La query 3) è join-equivalente alla query q

Sia $T(\underline{a}, b, c, d)$ una tabella usata nel FROM di una query q . Affinché q non generi record duplicati, è sufficiente che il SELECT di q contenga l'attributo 'a' oppure, se q fa uso di GROUP BY, un sottoinsieme proprio degli attributi di raggruppamento.

- ☐ a. Vero
- ☐ b. Falso

Dato lo schema di database usato a lezione:

PAZIENTE(CodFiscale, Nome, Cognome, Sesso, DataNascita, Citta, Reddito)

MEDICO(Matricola, Nome, Cognome, Specializzazione, Citta, Parcella)

VISITA(Paziente, Medico, Data, Mutuata)

che cosa restituisce la seguente query?

```
WITH Conti AS (  
  SELECT Paziente, Medico,  
         COUNT(*) AS C  
  FROM Visita  
  GROUP BY Paziente, Medico  
)  
SELECT Paziente, Medico,  
       RANK() OVER (  
         PARTITION BY Paziente  
         ORDER BY C DESC  
       ) AS Pos  
FROM Conti;
```

- ☐ a. Nessuna delle altre alternative è corretta
- ☐ b. Genera una classifica in cui, per ogni paziente p , un medico m è tanto più in alto in classifica quante più volte p è stato visitato da m
- ☐ c. Genera una classifica in cui, per ogni paziente p , un medico m è tanto più in alto in classifica quanto più è piccolo il numero di visite che p ha effettuato con m
- ☐ d. Genera una classifica in cui, per ogni medico m , un paziente p è tanto più in alto in classifica quante meno volte m ha visitato p

Siano dati lo schema di relazione $R(A,B,C,D,E,F,G,H,I,J)$ ed il relativo insieme di dipendenze funzionali $F = \{ABD \rightarrow E, AB \rightarrow G, B \rightarrow F, C \rightarrow H, C \rightarrow I, G \rightarrow J\}$. Quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☐ F è una copertura minimale.
- ☐ A è un attributo estraneo in almeno una dipendenza funzionale.
- ☐ Nessuna delle altre alternative è vera.
- ☐ F contiene almeno una dipendenza ridondante.

Quale delle seguenti alternative descrive una differenza tra COUNT(*) e COUNT(DISTINCT col) in MySQL, dove col è il nome di una colonna della tabella espressa nel FROM?

- ☐ a. COUNT(*) esclude le righe in cui almeno una colonna è NULL, mentre COUNT(DISTINCT col) le include tutte
- ☐ b. COUNT(*) conta tutte le righe, anche se contengono valori NULL, mentre COUNT(DISTINCT col) conta i valori diversi di col ad esclusione dei valori NULL
- ☐ c. COUNT(DISTINCT col) restituisce sempre un valore minore o uguale a quello di COUNT(*)
- ☐ d. Nessuna delle altre alternative è corretta

Quale delle seguenti affermazioni sulla clausola GROUP BY è corretta?

- ☐ a. Ogni colonna nella clausola SELECT deve essere aggregata
- ☐ b. Nessuna delle altre alternative è corretta
- ☐ c. La clausola GROUP BY può essere usata solo insieme alla clausola HAVING
- ☐ d. Data una query con GROUP BY, questa restituisce un insieme di record rappresentativi per ciascun gruppo

Sia $R(A,B,C,D)$ uno schema di relazione. Usando le regole di inferenza di Armstrong, quale delle seguenti derivazioni è falsa?

Scegli un'alternativa:

- ☐ $\{AB \rightarrow D\} \models ABC \rightarrow D$
- ☐ $\{C \rightarrow B, A \rightarrow D\} \models AC \rightarrow B$
- ☐ Nessuna delle altre alternative è falsa
- ☐ $\{AB \rightarrow D, B \rightarrow C\} \models AC \rightarrow D$

Dato lo schema di database usato a lezione:

PAZIENTE(CodFiscale, Nome, Cognome, Sesso, DataNascita, Citta, Reddito)

MEDICO(Matricola, Nome, Cognome, Specializzazione, Citta, Parcella)

VISITA(Paziente, Medico, Data, Mutuata)

cosa restituisce la seguente query?

```
SELECT DISTINCT P.Nome, P.Cognome  
FROM Paziente P  
      INNER JOIN  
      Visita V ON P.CodFiscale = V.Paziente  
WHERE V.Mutuata = 0;
```

- ☐ a. Nessuna delle altre alternative è corretta
- ☐ b. Nome e cognome dei pazienti che hanno almeno una visita non mutuata
- ☐ c. Nome e cognome dei pazienti che non hanno effettuato visite non mutate
- ☐ d. Nome e cognome dei pazienti che non hanno mai effettuato visite mutate

In MySQL, quale delle seguenti alternative descrive una differenza tra LEFT OUTER JOIN e INNER JOIN?

- ☐ a. LEFT OUTER JOIN può restituire record in cui tutte le colonne della tabella di destra assumono valore NULL, mentre ciò non può mai accadere con INNER JOIN.
- ☐ b. LEFT OUTER JOIN restituisce solo i record della tabella di sinistra per i quali esiste almeno un record della tabella di destra che soddisfa il predicato di join, mentre INNER JOIN restituisce tutti i record da entrambe le tabelle
- ☐ c. Nessuna delle altre alternative è corretta
- ☐ d. INNER JOIN ottiene lo stesso risultato di un LEFT OUTER JOIN seguito da un'opportuna clausola WHERE sulle colonne della tabella di sinistra, volta a eliminare i record di troppo che LEFT OUTER JOIN preserva

Si consideri lo schedule $S = w_2(x)r_1(x)w_1(x)r_2(x)$, composto dalle transazioni T_1 e T_2 . Quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☐ S è view-equivalente allo schedule seriale T_1T_2 .
- ☐ Nessuna delle altre affermazioni è vera.
- ☐ S è view-equivalente allo schedule seriale T_2T_1 .
- ☐ S non è view-serializzabile.

Quale delle seguenti affermazioni sulla forma normale di Boyce-Codd (BCNF) è vera?

Scegli un'alternativa:

- ☐ Se uno schema di relazione è in BCNF allora ha un'unica chiave.
- ☐ Determinare se uno schema di relazione in BCNF preserva le dipendenze funzionali ha complessità polinomiale.
- ☐ Se uno schema di relazione è in 3NF allora è in BCNF.
- ☐ Nessuna delle altre affermazioni è vera.

Sia $R(A,B,C,D)$ uno schema di relazione con $F=\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Data la decomposizione di R in $R_1(A,B)$, $R_2(B,C)$, $R_3(C,D)$, quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☐ La decomposizione preserva i dati ma non le dipendenze funzionali.
- ☐ Nessuna delle altre alternative è vera.
- ☐ La decomposizione preserva le dipendenze funzionali ma non i dati.
- ☐ La decomposizione preserva i dati e le dipendenze funzionali.

Dato lo schema di database usato a lezione:
PAZIENTE(CodFiscale, Nome, Cognome, Sesso, DataNascita, Citta, Reddito)
MEDICO(Matricola, Nome, Cognome, Specializzazione, Citta, Parcella)
VISITA(Paziente, Medico, Data, Mutuata)

e la richiesta:
Selezionare il codice fiscale dei pazienti che, quest'anno, hanno speso più di 500 Euro per visite effettuate da medici di ciascuna di almeno due città diverse, e qual è stato il numero di medici che hanno visitato questi pazienti in ogni specializzazione, considerando tutte le città nel complesso.

Quale delle seguenti alternative contiene una query che esprime la richiesta in MySQL?

- ☐ a. Nessuna delle altre alternative è corretta
- ☐ b.

```
WITH PazientiValidi AS (  
  SELECT V.Paziente  
  FROM Visita V  
    INNER JOIN  
      Medico M ON V.Medico = M.Matricola  
  WHERE YEAR(V.Data) = YEAR(CURRENT_DATE)  
  GROUP BY V.Paziente, M.Citta  
  HAVING SUM(M.Parcella) > 500  
)  
SELECT V.Paziente,  
       M.Specializzazione,  
       COUNT(DISTINCT M.Matricola)  
FROM Visita V  
  INNER JOIN  
    Medico M ON V.Medico = M.Matricola  
WHERE YEAR(V.Data) = YEAR(CURRENT_DATE)  
  AND V.Paziente IN (  
    SELECT Paziente  
    FROM PazientiValidi  
    GROUP BY Paziente  
    HAVING COUNT(*) >= 2  
  )  
GROUP BY V.Paziente, M.Specializzazione, M.Citta;
```
- ☐ c.

```
WITH PazientiValidi AS (  
  SELECT V.Paziente  
  FROM Visita V  
    INNER JOIN  
      Medico M ON V.Medico = M.Matricola  
  WHERE YEAR(V.Data) = YEAR(CURRENT_DATE)  
  GROUP BY V.Paziente, M.Citta  
  HAVING SUM(M.Parcella) > 500  
)  
SELECT V.Paziente,  
       M.Specializzazione,  
       COUNT(DISTINCT M.Matricola)  
FROM Visita V  
  INNER JOIN  
    Medico M ON V.Medico = M.Matricola  
WHERE YEAR(V.Data) = YEAR(CURRENT_DATE)  
  AND V.Paziente IN (  
    SELECT Paziente  
    FROM PazientiValidi  
    GROUP BY Paziente  
    HAVING COUNT(*) >= 2  
  )  
GROUP BY V.Paziente, M.Specializzazione;
```
- ☐ d.

```
WITH PazientiValidi AS (  
  SELECT DISTINCT V1.Paziente  
  FROM Visita V1  
    INNER JOIN  
      Medico M1 ON V1.Medico = M1.Matricola  
    INNER JOIN  
      Visita V2 ON V1.Paziente = V2.Paziente  
    INNER JOIN  
      Medico M2 ON V2.Medico = M2.Matricola  
  WHERE YEAR(V1.Data) = YEAR(CURRENT_DATE)  
    AND YEAR(V2.Data) = YEAR(CURRENT_DATE)  
    AND M1.Citta <> M2.Citta  
  GROUP BY V1.Paziente  
  HAVING SUM(M1.Parcella) > 500  
    AND SUM(M2.Parcella) > 500  
)  
SELECT V.Paziente,  
       M.Specializzazione,  
       COUNT(DISTINCT M.Matricola)  
FROM Visita V  
  INNER JOIN  
    Medico M ON V.Medico = M.Matricola  
  NATURAL JOIN  
    PazientiValidi PV  
WHERE YEAR(V.Data) = YEAR(CURRENT_DATE)  
GROUP BY V.Paziente, M.Specializzazione;
```

In una stored procedure MySQL, a cosa serve la clausola DECLARE CONTINUE HANDLER FOR NOT FOUND?

- ☐ a. A gestire gli errori generici che causano l'interruzione immediata della procedura, come divisioni per zero o violazioni di vincoli
- ☐ b. Nessuna delle altre alternative è corretta
- ☐ c. È tipicamente usata per intercettare la situazione in cui un'istruzione FETCH non restituisce record
- ☐ d. A indicare che, in caso di assenza di corrispondenza in una SELECT INTO, la procedura deve saltare il blocco corrente e terminare senza errore

Quale delle seguenti affermazioni riguardo alle stored procedure in MySQL è corretta?

- ☐ a. Una stored procedure può essere caratterizzata da più parametri IN e al più un parametro OUT
- ☐ b. Una variabile dichiarata con DECLARE all'interno di una stored procedure può essere usata anche all'esterno della procedura
- ☐ c. Un parametro OUT deve essere sempre oggetto di un assegnamento eseguito tramite un'istruzione SET o SELECT INTO
- ☐ d. Nessuna delle altre alternative è corretta

Quale delle seguenti caratteristiche deve essere posseduta da uno scheduler?

Scegli un'alternativa:

- ☐ Deve poter verificare la proprietà di view-serializzabilità di uno schedule in modo efficiente.
- ☐ Nessuna delle altre alternative è corretta.
- ☐ Deve poter verificare la proprietà di conflict-serializzabilità di uno schedule in modo efficiente.
- ☐ Deve poter risolvere tutte le anomalie di uno schedule in modo efficiente.

Dato il seguente schema di relazione $R(A,B,C,D,E,H)$ e il seguente insieme di dipendenze funzionali $F = \{ AB \rightarrow CD, C \rightarrow E, ABC \rightarrow D \}$. Quale delle seguenti decomposizioni di R è in 3NF?

Scegli un'alternativa:

- ☐ $R(A,B,C,D,E,H)$
- ☐ $R_1(A,B,C,D), R_2(C,E), R_3(A,B,H)$
- ☐ $R_1(A,B,C,D,E), R_2(C,H)$
- ☐ Nessuna delle decomposizioni indicate è in 3NF.

Quale delle seguenti alternative riguardo all'aggiornamento (UPDATE) di una tabella T in MySQL è falsa?

- ☐ a. MySQL impedisce di fare riferimento direttamente alla tabella target T in una UPDATE se tale riferimento è nel FROM
- ☐ b. Nessuna delle altre alternative è falsa
- ☐ c. Non è possibile aggiornare la tabella T in base a una condizione che coinvolge record della stessa tabella T
- ☐ d. È possibile aggiornare la tabella T in base a una condizione che coinvolge record della stessa tabella T usando una derived table che legge da T